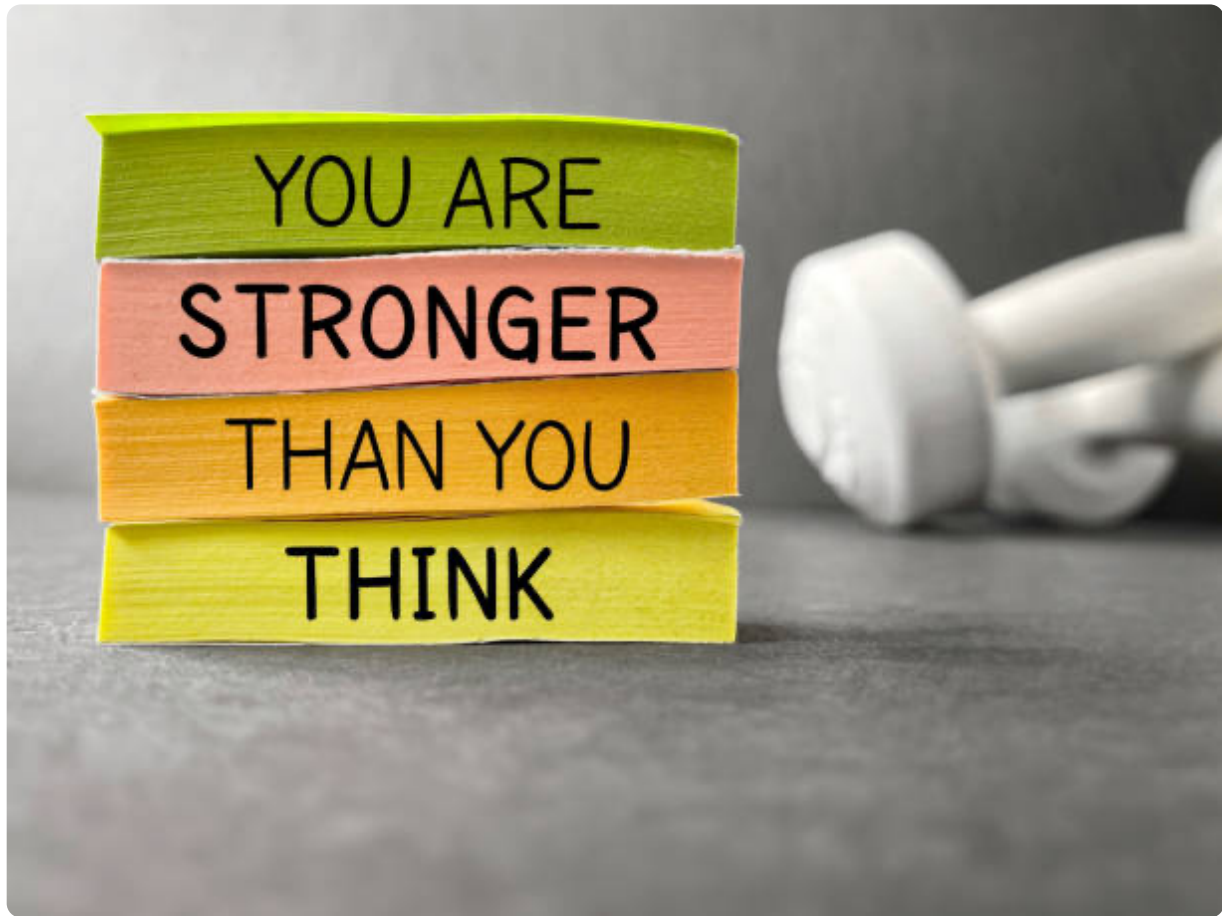




Array - Carry forward & Subarrays

TABLE OF CONTENTS

1. Count 'a-g' pairs
2. Sub-arrays
3. Print a Subarray
4. Print all Subarrays
5. Min-Max





Count 'a-g' pairs

< Question > : Given a string *s* of lowercase characters, return the count of pairs (*i*, *j*) such that $i < j$ and *s*[*i*] is 'a' and *s*[*j*] is 'g'.

str → a b e g a g
0 1 2 3 4 5
0,3 0,5
4,5
ans = 3

str → a e g d g a g
0 1 2 3 4 5 6
0,2 0,4 0,6
5,6
ans = 4

str → b c a g g a a g
0 1 2 3 4 5 6 7
2,3
2,7
2,7
5,7
6,7
ans = 5

 **BF Idea**

For every 'a', we need to find the count of g's on the right side of a. So, we need to have nested loops.

```
count = 0
for (i = 0; i < n; i++) {
    for (j = i + 1; j < n; j++) {
        if (s[i] == 'a' &&
            s[j] == 'g') {
            count++
        }
    }
}
```

TC: $O(n^2)$



(take whatever you calculated to the next index)



Idea

Carry Forward

str → b c a g g a a g

0 1 2 3 4 5 6 7

A 'g' will form a pair with ALL 'a' on its left

	0	1	2	3	4	5	6	7
	b	c	a	g	g	a	a	g
count_a	0	0	1	1	1	2	3	3
ans	0	0	0	1	2	2	2	5

count_a = 0 , ans = 0

for (i=0; i<n; i++) {

if (s[i] == 'a') {

count_a ++

}

else if (s[i] == 'g') {

ans += count_a

}

}
return ans

TC: O(N)

SC: O(1)



Subarrays

Definition

A subarray is a **contiguous** part of an array. It is formed by selecting a range of elements from the array. A subarray can have one or more elements and must be a contiguous part of the original array.

4 1 2 3 -1 6 9 8 12

0 1 2 3 4 5 6 7 8

Q → 4 -1 6 9 is sub-array?

✗

Q → 9 8 12 is sub-array?

✓

Q → 4 8 12 4 is sub-array?

✗

Q → 9 is sub-array?

✓



Example: $\text{arr}[] \rightarrow [2 \ 4 \ 1 \ 6 \ -3 \ 7 \ 8 \ 4]$

- a. [1, 6, 8] ✗
- b. [1, 4] ✗
- c. [6, 1, 4, 2] ✗
- d. [7, 8, 4,] ✓



Representation of a subarray

Representation of a Subarray

A Subarray can be uniquely represented in following ways:

- By specifying the start and end index of the subarray.
- By specifying the start index and length of the subarray.

If we consider the same array from the above example,

0	1	2	3	4	5	6	7	8
4	1	2	3	-1	6	9	8	12

The subarray 2, 3, -1, 6 can be represented as

- the range of elements starting at index 2 and ending at index 5 (0-based indexing).
- the range of elements having length of 4 with start index as 2.

l e
0 0, 1, 2, 3, ..., n-1



Total number of subarrays

[4 2 10 3 12 -2 15]
 0 1 2 3 4 5 6

$N = 7$

No. of subarrays starting from index 0 =

5
0

$0, 1, 2, 3, 4, 5, 6 = 7 (N)$

No. of subarrays starting from index 1 =

1

$1, 2, 3, 4, 5, 6 = 6 (N-1)$

No. of subarrays starting from index 2 =

$N-2$

No. of subarrays starting from index 3 =

$N-3$

...

...

No. of subarrays starting from index $n-1$ =

1

$1 + 2 + 3 + \dots + n =$

$$\frac{N(N+1)}{2}$$



< **Question** > : Given an array, si and ei. Print from si to ei.

$$si \leq ei$$

arr $\rightarrow$ [4 2 10 3 12 -2 15] si = 2, ei = 5
 0 1 2 3 4 5 6

• void printSubarray(arr, si, ei) {

for (i = si ; i $\leq$ ei ; i++) $\swarrow$
 print (arr[i])
}

}

print 1 subarray $\rightarrow$ T.C $O(N)$



< Question > : Print all the possible sub-arrays of the given array.

[5, 7, 3, 2]

0 1 2 3

O/P - [5]

[5, 7]

[5, 7, 3]

[5, 7, 3, 2]

[7]

[7, 3]

[7, 3, 2]

[3]

[3, 2]

[2]



Idea

To solve this problem, we can generate all possible subarrays of the array using two nested loops.

$TC: O(N^3)$



```
for (start = 0; start < n; start++) {  
    for (end = start; end < n; end++) {  
        printSubarr(arr, start, end)  
    }  
}
```

0 1 2 3
5 7 3 2

start	end	
0	0	5
0	1	5 7
0	2	5 7 3
0	3	5 7 3 2
1	1	7
1	2	7 3
1	3	7 3 2
2	2	3
2	3	3 2
3	3	2



Min Max

< Question > : Given an array of N integers, return the length of smallest subarray which contains both maximum and minimum elements of the array.

$1 \leq N \leq 10^6$

arr[] → [2 2 6 4 5 1 5 2 6 4 1]
 0 1 2 3 4 5 6 7 8 9 10

max = 6

min = 1

ans = 3

arr[] → [1 2 3 1 3 4 6 4 6 3]
 0 1 2 3 4 5 6 7 8 9

max = 6

min = 1

ans = 4

arr[] → [8 8 8 8 8 8]
 0 1 2 3 4 5

max = 8

min = 8

ans = 1



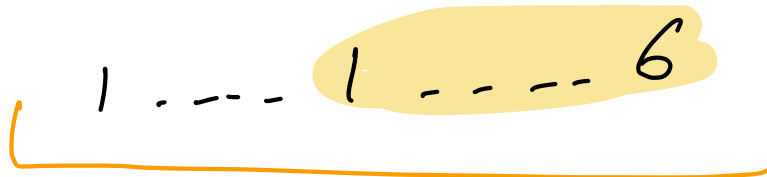
Idea

Check all subarrays

TC: $O(N^3)$



Observation



1. There must be exactly one occurrence of min & max element.

[min min - - - - max]

3 4 5 6 - - - - 1 3 4 5

2. Min and max elements should be the end point of subarray.

3. case-1: [min - - - - max]

case-2: [max - - - - min]

arr[] $\rightarrow$ [⁰2 ¹2 ²6 ³4 ⁴5 ⁵1 ⁶5 ⁷2 ⁸6 ⁹4 ¹⁰1]

} min = 1
max = 6

last_min -1 -1 -1 -1 -1 5 5 5 5 5 10
= -1

last_max -1 -1 2 2 2 2 2 2 8 8 8
= -1

ans = 3



min_val , max_val

last_min = -1

last_max = -1

ans = INT_MAX

↳ biggest value
of integer
datatype

```
for (i=0; i<n; i++) {
```

```
    if (arr[i] == minVal) {
```

```
        last_min = i
```

```
        if (last_max != -1) {
```

```
            | ans = min(ans, i - last_max + 1)
```

```
        }
```

```
    }
```

```
    if (arr[i] == maxVal) {
```

```
        last_max = i
```

```
        if (last_min != -1) {
```

```
            | ans = min(ans, i - last_min + 1)
```

```
        }
```

```
    }
```

```
}
```

TC: $O(N)$

SC: $O(1)$

return ans.

ans = ~~4~~ 3

max val = 6

min val = 1

arr[] → [⁰2 | ¹2 | ²6 | ³4 | ⁴5 | ⁵1 | ⁶5 | ⁷2 | ⁸6 | ⁹4 | ¹⁰1]

last_max

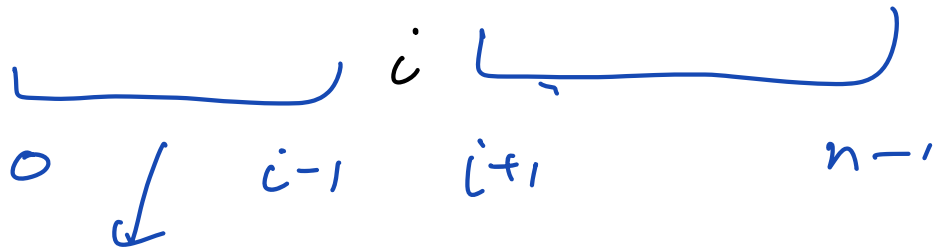
= -1

-1 -1 2 2 2 2 2 2 8 8 8

last_min

= -1

-1 -1 -1 -1 -1 5 5 5 5 5 10



$$pf[i-1] = pf[n-1] - pf[i]$$

$$pf[i-1] + pf[i] = pf[n-1]$$